

Documentation for the SRN Data Dashboard

Sustainability Reporting Navigator

2026-05-14

This whitepaper documents the automated ESG indicator extraction pipeline of the Sustainability Reporting Navigator (SRN). The pipeline retrieves and extracts a predefined set of 25 ESG indicators from firms’ annual and sustainability reports using a hybrid retrieval strategy combining semantic similarity search and keyword matching, followed by structured LLM extraction. We describe the design choices, retrieval parameters, and data schema. For technical questions and licensing inquiries, please reach out via email at hello@snav.com.

Table of contents

Transparency	1
Textual characteristics	2
Environmental and social topics	2
Performance	3
Data points	3
Pipeline setup	4
Pipeline performance	6
API accessibility	7
References	10

Transparency

The *transparency* dimensions captured by the SRN dataset describe **how** firms report rather than **what** they report. They are derived from the same annual and sustainability reports used for the quantitative ESG indicators (see [Performance](#)), but rather than extracting individual numbers, we measure textual properties of each document — its length, readability, topic coverage, and the specificity with which it discusses environmental and social issues. Reports are first parsed with Docling ([Deep Search Team 2024](#)) and then processed with Python at the sentence level. The downstream language model used to classify environmental and social passages was originally developed for Donau et al. ([2024](#)); we adopt their label set and decision thresholds without modification, and refer the reader to that paper for full methodological detail.

In what follows we summarise the two transparency dimensions exposed through the dashboard: a set of report-level textual characteristics (Section [Textual characteristics](#)) and a sentence-level classifier that locates passages addressing environmental and social topics (Section [Environmental and social topics](#)).

Textual characteristics

For each (firm, year, report) we compute a small number of report-level features describing the document's overall structure and writing style. They are stored alongside the extracted ESG indicators and surface in the dashboard as a single row per report. The features fall into three groups:

- **Length and structure.** Total token count, sentence count, average sentence length, and the number of pages devoted to environmental and social topics (derived from the sentence-level classifier introduced in the next subsection).
- **Readability.** [TODO: confirm which scores are computed — Flesch Reading Ease, Flesch–Kincaid Grade Level, Gunning Fog, or a combination. Note the tokenizer / sentence splitter used so the numbers are reproducible.]
- **Boilerplate and specificity.** [TODO: describe how boilerplate is identified (e.g., cross-firm n-gram overlap, repeated stock phrases) and how a specificity score is assigned to each report, citing the underlying method.]

[TODO: short table listing the exact column names exposed in the dashboard / API, with units and a one-line description per column. This mirrors Table~?? for the quantitative side.]

Environmental and social topics

At the sentence level, we tag each parsed sentence with one or more topic labels covering the environmental and social pillars of the European Sustainability Reporting Standards. The classifier is the supervised model trained in Donau et al. (2024); we run it sentence-by-sentence over the Docling output of both the annual and sustainability report. Its output is a long table with one row per (document, sentence) carrying the predicted label(s), the model's confidence score, and the original page number, which allows users to drill from any aggregate transparency statistic back to the source passage.

[TODO: enumerate the topic labels surfaced in the dashboard (e.g., climate change, pollution, water and marine resources, biodiversity, circular economy, own workforce, workers in the value chain, affected communities, consumers and end-users — and any social/governance labels you actively expose).]

[TODO: one paragraph on aggregation — how sentence-level predictions roll up to the report-level “share of text on topic X” and “number of pages on topic X” features used in the dashboard, including the decision threshold applied to the model's score.]

(TODO: one paragraph on validation — how the classifier was evaluated against a human-labelled sample, with the headline precision / recall numbers reported in [Donau et al. 2024](#) and any subsequent re-validation on the SRN corpus.)

Performance

This section documents how the SRN extracts the **quantitative** ESG indicators from firms' annual and sustainability reports and how accurately it does so. We first introduce the list of indicators currently in scope (Section [Data points](#)) and the rationale for the selection, then walk through the extraction pipeline end-to-end (Section [Pipeline setup](#)), report its accuracy on a human-reviewed subset of the most recent public release (Section [Pipeline performance](#)), and finally describe how the resulting dataset is exposed to downstream researchers (Section [API accessibility](#)).

Data points

Our list of datapoints covers all environmental, social, and governance (ESG) dimensions and is rooted in the European Sustainability Reporting Standards (ESRS, as prescribed by the Corporate Sustainability Reporting Directive). All inputs underlying the figures and tables of this document are publicly available and downloaded at render time; the snippet below pulls the indicator schema and the per-indicator performance snapshot for release v1.2, both served by the SRN community page without authentication.

```
"""Fetch every public input that backs this document.

All sources live on the SRN community release page and require no API
token. Already-downloaded files are reused so re-rendering stays fast.
"""

import json, re, urllib.request
from pathlib import Path

RELEASE      = "v1.2"
BASE         = f"https://lab.srnnav.com/p/main-esrs-kpi/{RELEASE}"
RELEASE_JSON = Path("release_v1_2.json")          # companies, fields, values
SNAPSHOT     = Path("performance_snapshot.json")  # per-indicator accuracy

def _http_get(url: str) -> bytes:
    req = urllib.request.Request(url, headers={"User-Agent": "srn-whitepaper/1.0"})
    return urllib.request.urlopen(req, timeout=30).read()

def _balanced(text: str, start: int) -> str:
    """Return text[start..matching '}']. start must point at an opening '{'."""
    depth, in_str, q, esc = 0, False, None, False
    for i in range(start, len(text)):
        c = text[i]
        if in_str:
```

```

        if esc:          esc = False
        elif c == "\\":  esc = True
        elif c == q:     in_str = False
    else:
        if c in ('"', "'"): in_str, q = True, c
        elif c == "{":     depth += 1
        elif c == "}":
            depth -= 1
            if depth == 0:
                return text[start: i + 1]
    raise RuntimeError("unbalanced braces")

# 1) Release data (used as the source of truth for indicator metadata).
if not RELEASE_JSON.exists():
    RELEASE_JSON.write_bytes(_http_get(f"{BASE}/data.json"))

# 2) Performance snapshot. Embedded as a devalue-serialised JS object literal
#    in the release page; we walk balanced braces and quote bare keys.
if not SNAPSHOT.exists():
    html = _http_get(BASE).decode("utf-8")
    anchor = html.find("performance:{")
    if anchor == -1:
        raise RuntimeError("performance snapshot not found on release page")
    js = _balanced(html, html.find("{", anchor))
    text = re.sub(r"([,])([A-Za-z_]\w*):", r'\1"\2":', js)
    SNAPSHOT.write_text(json.dumps(json.loads(text), indent=2))

for p in (RELEASE_JSON, SNAPSHOT):
    print(f"'cached ' if True else 'fetched'">{p} ({p.stat().st_size:,} bytes)")

```

```

cached release_v1_2.json  (829,548 bytes)
cached performance_snapshot.json  (8,712 bytes)

```

Pipeline setup

Our extraction pipeline builds on the design described by Forster et al. (2025). For each firm-year, we extract a predefined set of 25 ESG indicators from the firm's annual report and sustainability report. Figure 1 summarises the pipeline end-to-end.

Text, tables, and figures of the source documents are first parsed and then chunked — that is, split into smaller units — using Docling (Deep Search Team 2024). To make chunks quickly retrievable,

Indicator	Unit
<i>Environmental</i>	
Total energy consumption	MWh
Total energy consumption from fossil sources	MWh
Total energy consumption from renewable sources	MWh
Scope 1 greenhouse gas emissions	tCO2eq
Scope 2 greenhouse gas emissions (location-based)	tCO2eq
Scope 2 greenhouse gas emissions (market-based)	tCO2eq
Total scope 3 greenhouse gas emissions	tCO2eq
Total emissions (location-based)	tCO2eq
Total emissions (market-based)	tCO2eq
Total water consumption	m3
Total use of land	m2
Total weight of waste generated	t
Total weight of non-recycled waste	t
Share of non-recycled waste to total waste	%
<i>Social</i>	
Total number of employees	—
Total number of female employees	—
Share of employees 50+	%
Rate of employee turnover	%
Percentage of total employees covered by collective bargaining agreements	%
Share of women in top management	%
Rate of recordable work-related accidents	—
Gender pay gap	%
Annual total remuneration ratio	%
Number of severe human rights issues and incidents	—
<i>Governance</i>	
Total number confirmed incidents of corruption or bribery	—

Table 1: ESG indicators extracted by the pipeline.

we embed each chunk with Mistral’s text embedding model, `mistral-embed-2312` (Mistral AI 2023). The resulting numerical vectors are stored in a cosine-similarity index that serves as the backbone for retrieval.

For each of the 25 indicators, retrieval combines two complementary signals in a single hybrid step. Mistral Large first turns the indicator definition into a natural-language search query, which is embedded with Mistral Embed and submitted to the pgvector index alongside a BM25 lexical query over the chunk text. The two ranked lists are merged via reciprocal rank fusion (RRF) with weights 0.7 for the vector channel and 0.3 for the lexical channel, and the top five chunks (subject to a cosine-similarity floor of 0.70) are returned. This hybrid formulation is motivated by the observation that purely semantic retrieval occasionally misses tables or boilerplate passages that use the exact regulatory term, while pure keyword matching ignores synonymous formulations.

The retrieved chunks, together with a custom prompt and a structured output schema, are passed to the language model. The prompt specifies the indicator definition, the expected unit of measurement, and instructions for handling absent disclosures. The model returns the extracted value, its source page, a confidence score in $[0, 1]$, and a short justification. Outputs are validated in-flight against a Zod schema;

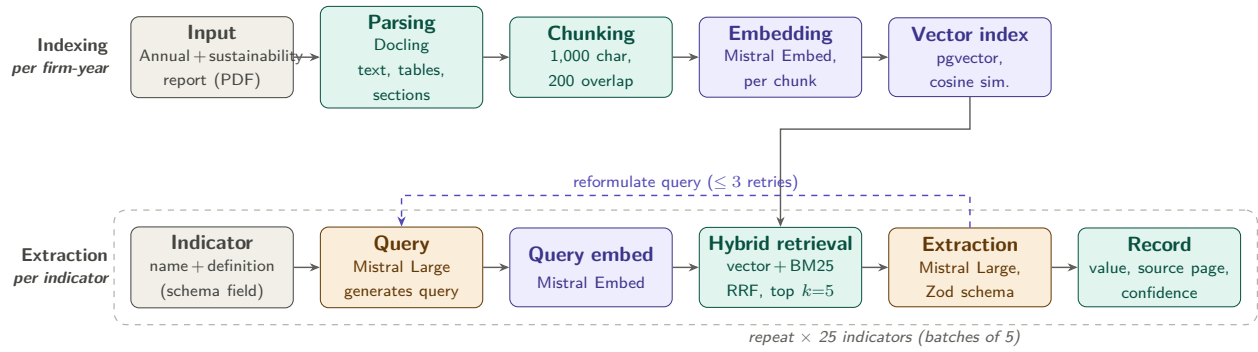


Figure 1: ESG extraction pipeline. **Top lane (indexing, one-off per firm-year)**: the annual and sustainability reports are parsed with Docling, split into overlapping chunks ($\sim 1,000$ characters, 200-character overlap, sentence-aware), embedded with Mistral Embed, and stored in a pgvector cosine-similarity index. **Bottom lane (extraction, per indicator)**: Mistral Large turns the indicator definition into a search query; the query is embedded; a hybrid retrieval combines vector similarity with BM25 lexical scoring via reciprocal rank fusion (RRF, weights 0.7/0.3) and returns the top five chunks; Mistral Large then produces a structured extraction validated against a Zod schema and stored together with its source-page provenance. If the model reports that the information is contained in the retrieved sources but cannot be extracted as-is, the query is reformulated and retrieval is retried (at most three times).

responses that violate the schema are discarded and logged rather than retried, to avoid propagating model errors downstream. If a response is well-formed but reports that the answer is present in the retrieved sources yet could not be extracted as a clean value, the query is reformulated by the model and retrieval is repeated, up to three times in total.

The loop runs over all 25 indicators per firm-year in batches of five and results are written to a relational table with one row per indicator–firm–year triplet. Each record retains the source-document identifier and page number, enabling manual verification against the original filing.

Pipeline performance

To assess extraction quality we evaluate every AI-extracted value against a human-verified ground truth. The evaluation set used here is the human-reviewed subset of public release v1.2 of the *Main ESRS KPI* project, covering 4,608 firm–indicator–year pairs for Euro Stoxx 50 and DAX 40 companies in the reporting years 2024 and 2025. Each pair was independently inspected by a trained reviewer who either accepted the AI’s output, replaced it with the correct number, or marked the indicator as not disclosed. Outcomes are partitioned into five mutually exclusive classes: *correct* (the AI value matches the reviewer’s value, including agreement on non-disclosure); *wrong* (both the AI and the reviewer extracted a number but the values disagree); *false negative* (the AI returned NOT_FOUND but the reviewer found a value); *false positive* (the AI returned a value but the reviewer marked the indicator as not disclosed); and *schema error* (the AI’s structured output failed Zod validation).

Aggregated across all 25 indicators, the pipeline reaches an accuracy of 72% on the reviewed subset

(3,322 of 4,608 pairs). Breaking this down by category, governance (one indicator) and social (75.0%) lie slightly above this mean while environmental indicators (69.8%) lie slightly below it, driven primarily by the harder Scope 1–3 greenhouse-gas emissions. Across error modes, *wrong* extractions (619 pairs, 13% of the reviewed set) are the largest single failure class, followed by *false negatives* (443 pairs, 10%); *false positives* (124, 3%) and *schema errors* (100, 2%) are comparatively rare.

False negatives concentrate on the greenhouse-gas indicators (Scope 1, Scope 2 location- and market-based, total emissions, and Scope 3), where reports frequently use bespoke wording, hide the value in a table footnote, or report partial sub-figures that confuse the retrieval step. The same indicators also produce many *wrong* extractions because firms disclose multiple consolidations (e.g., group-wide versus segment-level, with or without biogenic emissions) and the model occasionally selects a competing nearby figure. Conversely, employee counts, the share of female employees, and total water and land use achieve accuracies above 84%; these indicators are typically disclosed as a single, prominently placed number with a stable wording. The annual total remuneration ratio is the lone outlier on the low end (46%): firms report this metric inconsistently, often as a derived ratio with multiple definitions, which yields a high *wrong* rate that is not retrieval-bound but reflects genuine ambiguity in the underlying disclosure.

API accessibility

The extracted dataset is exposed through a small, versioned REST API at api.srnav.com. Each *release* is an immutable, citeable snapshot of the dataset — once published, its values, version label, and row count never change; superseded releases remain reachable and any subsequent retraction is recorded on the release itself. We recommend pinning research artifacts to an explicit version (e.g. v1.2) rather than to latest, so that downstream results remain reproducible when new releases are published. Interactive endpoint documentation, request/response schemas, and a try-it-out console are available at api.srnav.com, which serves a Swagger UI generated from the OpenAPI specification at [/openapi.json](https://openapi.json).

Access is gated by a personal API token, requested at srnav.com and passed in the standard Authorization: Bearer header. Tokens are scoped to the issuing account and can be rotated or revoked at any time; we ask consumers to keep them out of version control and out of figure source files. The two endpoints needed for most research workflows are:

GET /projects/{slug}/releases Lists all published releases for a public project, newest first, with publication and retraction timestamps and row counts. Used to discover the latest version programmatically and to track the dataset’s revision history.

GET /projects/{slug}/releases/{version} Returns the full release as JSON, where {version} accepts latest, v1, or v2.1. A CSV variant is available at the same path with a .csv suffix and produces a flattened, header-included file suitable for direct loading into a statistical package.

Each row of a release identifies a single firm–indicator–year observation and carries the published value together with the firm’s identifier, ISIN, country, sector, industry, and stock index membership; the

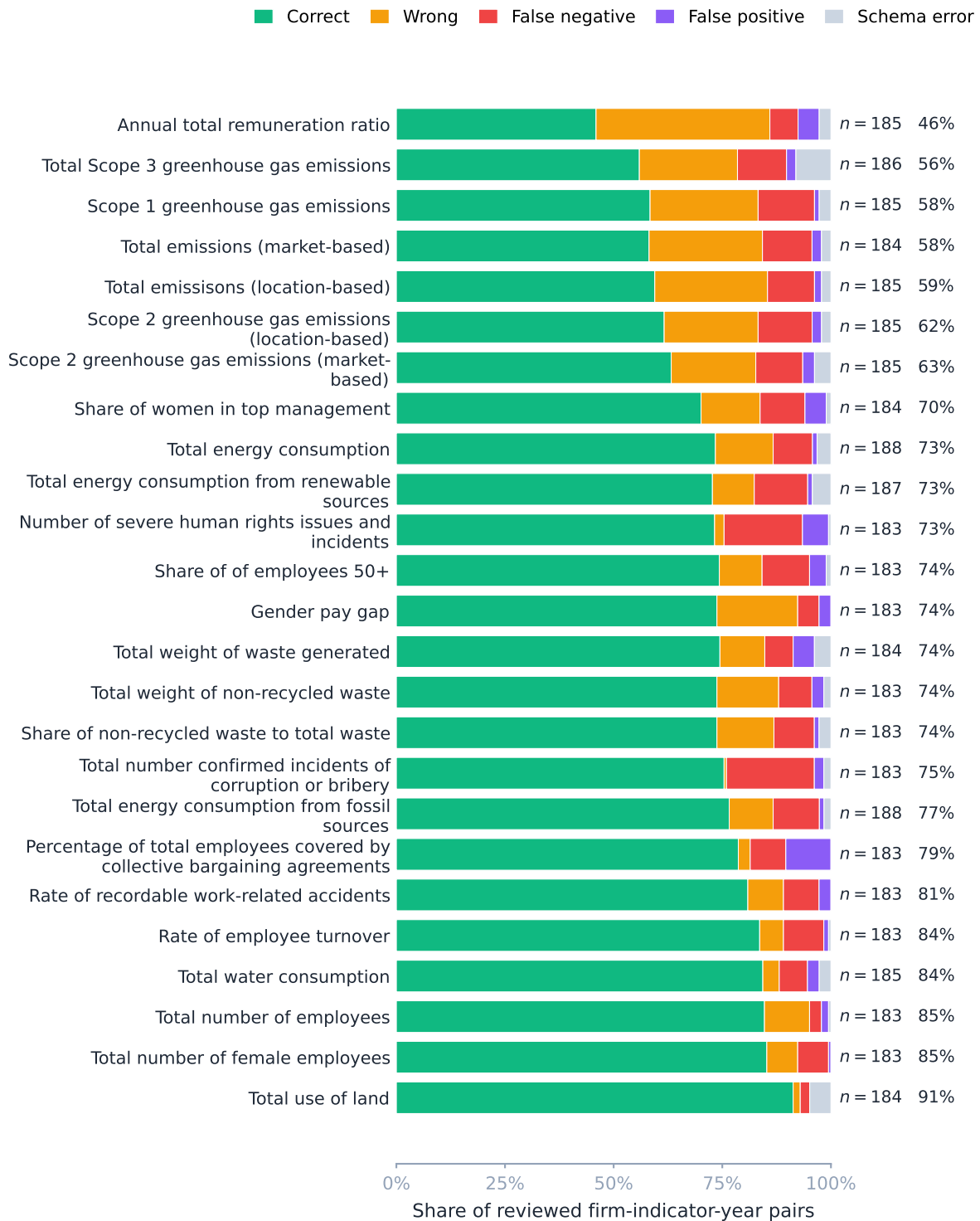


Figure 2: Per-indicator extraction accuracy on the human-reviewed subset of public release v1.2 (4,608 firm-indicator-year pairs covering Euro Stoxx 50 and DAX 40 companies in reporting years 2024–2025). Bars are sorted by accuracy. Each stacked bar shows the composition of the reviewed pairs into five outcomes; the rightmost annotation reports the number of reviewed pairs n and the per-indicator accuracy. The overall accuracy of the pipeline on this set is 72%.

indicator's slug, name, category, unit, and data type; the year; a source flag (human for reviewer-verified values, ai for AI-only extractions awaiting review); the source document identifier; and a page reference for traceability against the original filing.

A minimal request fetches the latest release:

```
curl -H "Authorization: Bearer $SRN_TOKEN" \  
  "https://api.srnav.com/projects/main-esrs-kpi/releases/latest"
```

The same payload is one line away from a tidy data frame in Python — the following snippet pins the request to a specific release and filters to the GHG indicators for two firms:

```
import os, requests, pandas as pd  
  
VERSION = "v1.2"  
headers = {"Authorization": f"Bearer {os.environ['SRN_TOKEN']}"}  
  
resp = requests.get(  
    f"https://api.srnav.com/projects/main-esrs-kpi/releases/{VERSION}",  
    headers=headers, timeout=30,  
)  
resp.raise_for_status()  
release = resp.json()  
  
df = pd.DataFrame(release["values"])  
df["value"] = pd.to_numeric(df["value"], errors="coerce") # NOT_FOUND -> NaN  
  
ghg = df[  
    df["field_slug"].str.startswith("ghg_") &  
    df["isin"].isin(["DE000A1EWWO", "DE0007164600"]) # Adidas, SAP  
)  
print(ghg[["company_name", "year", "field_slug", "value", "unit", "source"]])
```

The CSV variant is most convenient in R, where it can be read in a single call:

```
library(readr)  
  
token <- Sys.getenv("SRN_TOKEN")  
version <- "v1.2"  
  
df <- read_csv(  
    paste0("https://api.srnav.com/projects/main-esrs-kpi/releases/",
```

```

    version, ".csv"),
  col_types = cols(.default = col_character(), value = col_double()),
  na        = c("", "NA", "NOT_FOUND"),
  show_col_types = FALSE,
  guess_max = 1e5,
  # readr passes header arguments via `httr::add_headers` when the URL is fetched
  # by an internal call to httr; for explicit auth use the `httr2` recipe below.
)

```

For programmatic discovery of the most recent published release and for any workflow that requires per-request control (e.g. rate-limit handling or audit logging), `httr2` composes cleanly with the `releases` index:

```

library(httr2)

req <- request("https://api.srnnav.com/projects/main-esrs-kpi/releases") |>
  req_auth_bearer_token(Sys.getenv("SRN_TOKEN")) |>
  req_user_agent("srn-research/0.1")

releases <- resp_body_json(req_perform(req))
latest   <- Filter(\(r) is.null(r$retracted_at), releases)[[1]]$version
message("Pinned to ", latest)

```

We encourage users to cite the dataset by its release version (e.g., *Sustainability Reporting Navigator, Main ESRS KPI dataset, release v1.2, published 2026-05-14*) and, where possible, to record the release's `published_at` timestamp in supplementary materials so that subsequent retractions or revisions can be detected. Requests, feature suggestions, and bug reports are welcome by email at hello@srnav.com.

References

- Deep Search Team. 2024. “Docling Technical Report.” arXiv:2408.09869. IBM Research. <https://doi.org/10.48550/arXiv.2408.09869>.
- Donau, Charlotte-Louisa, Katharina Hombach, Inga Elise Meringdal, Maximilian A Müller, Thorsten Sellhorn, and Victor Wagner. 2024. “Targeting Transparency: Early Evidence on Mandatory Adoption of European Sustainability Reporting Standards.”
- Forster, Kerstin, Lucas Keil, Victor Wagner, Maximilian A Müller, Thorsten Sellhorn, and Stefan Feuerriegel. 2025. “Assessing Corporate Sustainability with Large Language Models: Evidence from Europe.” <https://ssrn.com/abstract=5361703>.
- Mistral AI. 2023. “Mistral Embed (mistral-embed-2312) – Model Card.” <https://docs.mistral.ai/models/model-cards/mistral-embed-23-12>.